

ndp_tests

Instrukcja całego projektu koncepcyjnego

Wersja 1.0 - 29 czerwca 2026 r.

1. Cel systemu

ndp_tests jest platformą dydaktyczną do udostępniania materiałów, przeprowadzania krótkich testów, prezentowania wyników oraz wspierania prowadzącego w analizie prób. Frontend jest przygotowany do statycznego hostowania na GitHub Pages. Funkcje wymagające ochrony danych, uwierzytelniania i zapisu centralnego muszą być realizowane przez oddzielny backend.

2. Role użytkowników

- Student - przegląda materiały, uruchamia test, udziela odpowiedzi i sprawdza wyniki.
- Prowadzący - zarządza zakresem testów, analizuje wyniki i pobiera raporty.
- Administrator - zarządza użytkownikami, uprawnieniami, konfiguracją i audytem w wersji produkcyjnej.

3. Architektura

Repozytorium ma strukturę monorepo. Frontend znajduje się w apps/web, opcjonalny backend w apps/api, a dokumentacja w docs. GitHub Actions buduje statyczny eksport Next.js i publikuje katalog wynikowy na GitHub Pages.

Najważniejsze ścieżki:

```
apps/web/app - strony i komponenty aplikacji
apps/web/config/design-tokens.css - kolory, typografia i wspólne wartości wyglądu
apps/web/public/downloads - publiczne pliki demonstracyjne
docs - instrukcje techniczne i wdrożeniowe
agents.md - obowiązkowe zasady pracy agentów AI
```

4. Uruchomienie lokalne

W katalogu głównym repozytorium wykonaj:

```
npm install
npm run dev -w apps/web
```

Aplikacja będzie dostępna pod lokalnym adresem wskazanym przez Next.js. Przed publikacją wykonaj statyczną kompilację:

```
npm run build:pages
```

5. Publikacja na GitHub Pages

- Utwórz repozytorium o nazwie ndp_tests.
- Wgraj zawartość paczki i ustaw główną gałąź main.
- W Settings Pages wybierz źródło GitHub Actions.
- Wykonaj push do gałęzi main.
- Sprawdź działanie strony pod adresem konta GitHub i ścieżką repozytorium.

6. Panel prowadzącego i eksporty

W wersji statycznej panel prowadzącego udostępnia demonstracyjne pliki CSV i PDF oraz niniejszą instrukcję. Dane te nie są pobierane z bazy. W wersji produkcyjnej raport powinien być generowany przez

backend dopiero po sprawdzeniu sesji, roli i przypisania prowadzącego do kursu.

7. Dostępność

- Stosuj semantyczny HTML i obsługę klawiaturą.
- Zapewnij widoczny fokus i kontrast zgodny z WCAG 2.2 AA.
- Nie przekazuj znaczenia wyłącznie kolorem.
- Sprawdzaj układ przy powiększeniu 200%.
- Nie używaj emotikon w interfejsie i dokumentacji.

8. System wizualny

Kolory, rozmiary czcionek, grubości tekstu, wysokości linii, promienie i cienie są zdefiniowane w pliku `apps/web/config/design-tokens.css`. Komponenty nie powinny zawierać powielonych stałych wizualnych. Style inline są dopuszczalne tylko dla wartości dynamicznych, takich jak procentowa szerokość paska postępu.

9. Bezpieczeństwo i dane osobowe

- Nie publikuj rzeczywistych wyników studentów w katalogu publicznym.
- Maskuj identyfikatory w widokach demonstracyjnych.
- Nie przechowuj sekretów w repozytorium.
- Telemetrii nie traktuj jako automatycznego dowodu niesamodzielności.
- Eksporty produkcyjne rejestruj w dzienniku audytowym.

10. Zasady rozwoju

Projekt stosuje KISS i zakaz nadmiernego inżynierowania. Najpierw należy wykorzystać istniejące komponenty i standardowe API platformy. Nowe biblioteki, warstwy i abstrakcje powinny być dodawane wyłącznie wtedy, gdy rozwiążą konkretny, aktualny problem i upraszczają utrzymanie.

11. Kryteria odbioru zmiany

- Kompilacja TypeScript kończy się bez błędów.
- Frontend przechodzi statyczny eksport.
- Nawigacja i nowe elementy działają z klawiatury.
- Dokumentacja odpowiada implementacji.
- Repozytorium nie zawiera nieoznaczonych danych demonstracyjnych ani danych rzeczywistych studentów.

12. Dalsze kroki produkcyjne

- Wdrożyć uwierzytelnianie i autoryzację ról.
- Podłączyć bazę danych i serwerową kontrolę czasu.
- Generować raporty na backendzie.
- Dodać dziennik audytowy i politykę retencji danych.
- Przeprowadzić testy dostępności, bezpieczeństwa i obciążenia.